



# JMDT Layer 2 Blockchain

The Technical White Paper

---

## Abstract

**JMDT** is the Truth Layer for all information - Restoring authenticity in digital infrastructure by privately verifying humans, and decentralising their data.

**Jupiter Meta Data Token Chain (JMDT)** is a modular, Ethereum-based **Layer 2 (L2) blockchain protocol** designed to address the scalability, privacy, and compliance limitations of traditional blockchain systems. Built with **Zero-Knowledge Proofs (ZKPs)**, **Decentralized Identity (DID)**, and our own proprietary **Asynchronous Validation Consensus (AVC)**, JMDT delivers a high-performance, privacy-preserving infrastructure tailored for both decentralized applications and enterprise-grade solutions.

At the core of JMDT is its **Layer 3 (L3) Enterprise Privacy Mesh**, a **Directed Acyclic Graph (DAG)-based architecture** that allows enterprises to process **private, high-throughput transactions** locally. These state transitions are **ZK-validated and cryptographically committed to L2**, ensuring global auditability without exposing raw data. This architecture provides organizations with full **data sovereignty**, while retaining the **integrity and verifiability of blockchain finality**.

JMDT's **dynamic rollup architecture** anchors L2 state changes to **Ethereum Layer 1**, optimizing for gas fees, block time, and settlement efficiency. All zk-proofs are generated via **Rust-based circuits** executed in the **RISC Zero zkVM**, offering reproducibility, auditability, and verifiable computation with STARK-level guarantees.

The protocol's **AVC consensus mechanism** combines:

- **VRF-based Buddy Node selection** with adaptive weights from Seed Nodes,
- **zk-proof integration** to strengthen validation integrity,
- a **Gossip protocol** for efficient propagation of blocks, votes, and metadata, and
- **CRDT-based conflict resolution** to guarantee convergence even under partitions.

This hybrid design achieves **scalable, asynchronous, and secure block finality** while avoiding the high communication and energy costs of traditional consensus models.

With native support for **DID-based Zero knowledge authentication**, **privacy-preserving queries**, **immutable storage**, and **tokenized governance using JMDT Utility Token**, JMDT is engineered to power next-generation decentralized applications and ecosystems.

By uniting **verifiability, modular enterprise layers, and cryptographic privacy**, Jupiter Meta Data Token stands as a forward-compatible, high-assurance blockchain protocol built for the decentralized future.

---

## 1. Introduction

**Jupiter Meta Data Token (JMDT)** is an **Ethereum-based Layer 2 (L2) blockchain protocol** built to deliver **scalable, privacy-preserving, and verifiably secure computation** for both enterprise and decentralized applications. By integrating **Zero-Knowledge Proofs (ZKPs)**, **Decentralized Identity (DID)**, and **AVC consensus model**, JMDT offers a **next-generation infrastructure** designed for **data sovereignty, modular deployment, and interoperability across Web3 ecosystems**.

JMDT's architecture spans across three layers:

- **L3 DAG Layer** for enterprise-grade, high-throughput data operations.
- **L2 Rollup Layer** for privacy-preserving execution and zk-based consensus.
- **L1 Finality Layer** for Ethereum anchoring and global settlement.

---

## 1.1 Why Jupiter Meta Data Token (JMDT)?

Conventional blockchains struggle with performance, privacy, and real-world compliance. JMDT addresses these systemic limitations through a tightly integrated architecture:

- **Zero-Knowledge Proofs (ZKPs)**: Enabling human Identity verification, private transaction validation, and 100% data privacy - through trustless infrastructure that does not reveal any information beyond the fact that the statement is true.
- **Decentralized Identity (DID)**: W3C-compliant, privacy-preserving authentication framework that verifies PII with on-chain records without accessing any PII whatsoever.
- **Layer 2 Scaling**: High-throughput zkRollup engine built using RISC Zero zkVM for verifiable Rust-based execution. Designed for Faster settlements and the ability to process large scale concurrent transactions in seconds.
- **AVC Consensus**:
  - **Scalable and secure block finality** through quorum-based buddy voting and asynchronous global tally
  - **Energy-efficient operation** by eliminating mining and token-weighted privilege
  - **Low-latency, fault-tolerant propagation** of consensus-critical data, and
  - **Seamless compatibility** with DAG-based Layer-3 enterprise extensions.

AVC is optimized for use cases requiring **privacy, auditability, and reliability** — including digital identity verification, decentralized finance applications and cross-organizational data collaboration.

- **Ethereum Compatibility**: L2 rollups seamlessly settle on Ethereum L1 using verifiable zk-proofs.
- **Immutable Ledger**: Ensures tamper-proof audit trails across all data and DAG activity.
- **Privacy-Preserving Queries**: Decentralized Identity based access with zk-proofed data capture, enabling GDPR-compliant data sharing.
- **Enterprise Mesh Model**: Supports interoperable DAGs per organization, enabling modular private ecosystems with shared L2 accountability.

---

## 1.2 Core Objectives

JMDT is the Truth Layer for all information on the internet - a single source truth for all of humanities data.

We are built to support completely Decentralised Identity verification, on-chain data validation and proof-based data sharing. This is how do it;

| Objective        | Description                                                                              |
|------------------|------------------------------------------------------------------------------------------|
| Scalability      | Achieves >2,000+ TPS via zkRollups and DAG batching                                      |
| Privacy          | End-to-end privacy via ZKPs and DID-based access controls                                |
| Security         | zkVM-based execution, AVC consensus, and stake-based accountability                      |
| Interoperability | Fully compatible with Ethereum, and Web3 SDKs                                            |
| Efficiency       | Bloom Filters reduce propagation redundancy; zkRollups optimize gas fees for L1 finality |
| Governance       | JMDT token enables on-chain DAO governance and validator staking                         |

### 1.3 A Future-Proof Blockchain Infrastructure

JMDT combines the rigor of zero-knowledge cryptography with the practical needs of enterprises and decentralized communities. With **zkRollups for privacy**, **DAGs for performance**, **DIDs for access control**, and **Ethereum for settlement**, JMDT offers a **layered, composable, and audit-ready blockchain foundation** for:

- Decentralized finance (DeFi)
- Web3 Data Marketplaces
- Identity management platforms
- AI-driven analytics pipelines
- Public sector and compliance-first applications

## 2. Vision and Mission

“Building the truth layer for human intelligence—where verified humans own their data, enterprises access verified insights, and privacy is absolute.”

*We are enabling a single source of truth for all human information.*

A **next-gen blockchain infrastructure** where **decentralized identities**, **zero-knowledge proofs**, and **verifiable computation** allow us to **redefine how data is owned, controlled and monetized** without compromising compliance **across digital platforms**.

-

*Privacy-first. Truth-verified. Human-owned. Decentralized by design.*

Currently, JMDT’s Web3 ecosystem has cryptographically verified over 20 million humans without exposing their identity. Allowing them to monetize authentic data that they actually own, and provide insights that previously enriched everyone but them.

Enterprises are in turn given access to this decentralised human intelligence with a verifiable chain of custody—from verified human and immutable blockchain storage to predictive insights that result in massive monetary value.

## 3. Technology Stack

### 1. Zero-Knowledge Proofs (ZKPs)

JMDT leverages **Zero-Knowledge Proofs (ZKPs)** to enable **privacy-preserving identity verifications, authentication, transactions, and computations** without revealing sensitive information. ZKPs allow users to verify transaction validity without exposing underlying data, ensuring security and confidentiality across financial, identity, and enterprise applications. JMDT's proving pipeline is built on zk-STARKs (Scalable Transparent Argument of Knowledge), generated via RISC Zero's zkVM. STARKs were selected over SNARK-based approaches specifically because they require no trusted setup and offer hash-based, quantum-resistant security guarantees — both material considerations for a protocol anchoring enterprise and identity data.

### 2. Decentralized Identity (DID)

JMDT incorporates **W3C-standardized Decentralized Identifiers (DIDs)** to provide users with **self-sovereign identities** that enable authentication across **dApps, DeFi, and enterprise services**. **DID-based authentication** ensures **secure, private access** while preventing unauthorized data exposure. This eliminates reliance on centralized identity providers and enhances compliance with **GDPR, HIPAA, and data sovereignty regulations**.

### 3. Layer 2 Scaling

JMDT uses **zk-rollups** and **immutable ledger-based state synchronization** to significantly enhance transaction throughput and efficiency. By bundling multiple transactions into a single proof before submitting them to **Ethereum (L1)**, JMDT achieves **lower gas fees, reduced congestion, and enhanced finality**, making it ideal for **DeFi, gaming, and high-frequency enterprise transactions**.

### 4. AVC Consensus Algorithm

The **AVC Consensus Algorithm** is a purpose-built framework for decentralized, high-throughput blockchain systems. It ensures **secure participation, adaptive fairness, and scalability** by combining verifiable randomness, zero-knowledge validation, and fault-tolerant data structures.

AVC integrates:

1. **VRF-based Buddy Node Selection** – A deterministic verifiable random function, seeded and weighted by Seed Node feedback, selects a unpredictable randomized buddy set each round. This ensures fairness, geo-diversity, and resistance to Sybil or cartel attacks.
2. **Zero-Knowledge Proof Integration** – zk-proofs strengthen block validation, allowing lightweight and privacy-preserving verification of transactions and state transitions without revealing sensitive data.
3. **Gossip Protocol** – A decentralized backbone for disseminating transactions, buddy votes, and zk-proofs efficiently across the JMDT Decentralized Node (JMDN) network.
4. **Bloom Filters** – Used to prevent duplicate or replayed transactions during gossip and validation, reducing bandwidth overhead.
5. **Conflict-Free Replicated Data Types (CRDT)** – Enable eventual consistency in buddy and network states (e.g., votes, digests, mempool views) without central coordination, ensuring convergence even under partitions.
6. **Append-Only Ledger** – Provides tamper-proof storage of finalized blocks, while malicious or invalid data is flagged and stored separately for auditability.

The consensus design delivers:

- **Scalable and secure block finality** through quorum-based buddy voting and asynchronous global tally,

- **Energy-efficient operation** by eliminating mining and token-weighted privilege,
- **Low-latency, fault-tolerant propagation** of consensus-critical data, and
- **Seamless compatibility** with DAG-based Layer-3 enterprise extensions.

AVC is optimized for use cases requiring **privacy, auditability, and reliability** — including digital identity, decentralized finance and cross-organizational data collaboration.

#### 5. Ethereum Interoperability

JMDT is **fully compatible with Ethereum smart contracts**, enabling **seamless integration with existing DeFi applications, NFT marketplaces, DAOs, and enterprise solutions**. Developers can build and deploy Solidity-based applications on JMDT while benefiting from **L2 scalability, lower gas costs, and enhanced security**.

#### 6. Efficient L1 Commitments

To ensure security and transparency, **all Layer 2 transactions on JMDT are committed to Ethereum (L1)** based on **dynamic optimization of gas fees, block times, and network congestion**. This **adaptive commitment strategy** balances cost-efficiency with **on-chain finality**, ensuring robust security without excessive transaction fees. *See Section 5.1 for the full EIP-1559 gas-aware submission protocol.*

#### 7. Immutable Storage

JMDT leverages **immutability-focused storage mechanisms**, to ensure **tamper-proof, cryptographically verifiable data recording**. This is critical for **auditable enterprise applications, supply chain tracking, and regulatory compliance**.

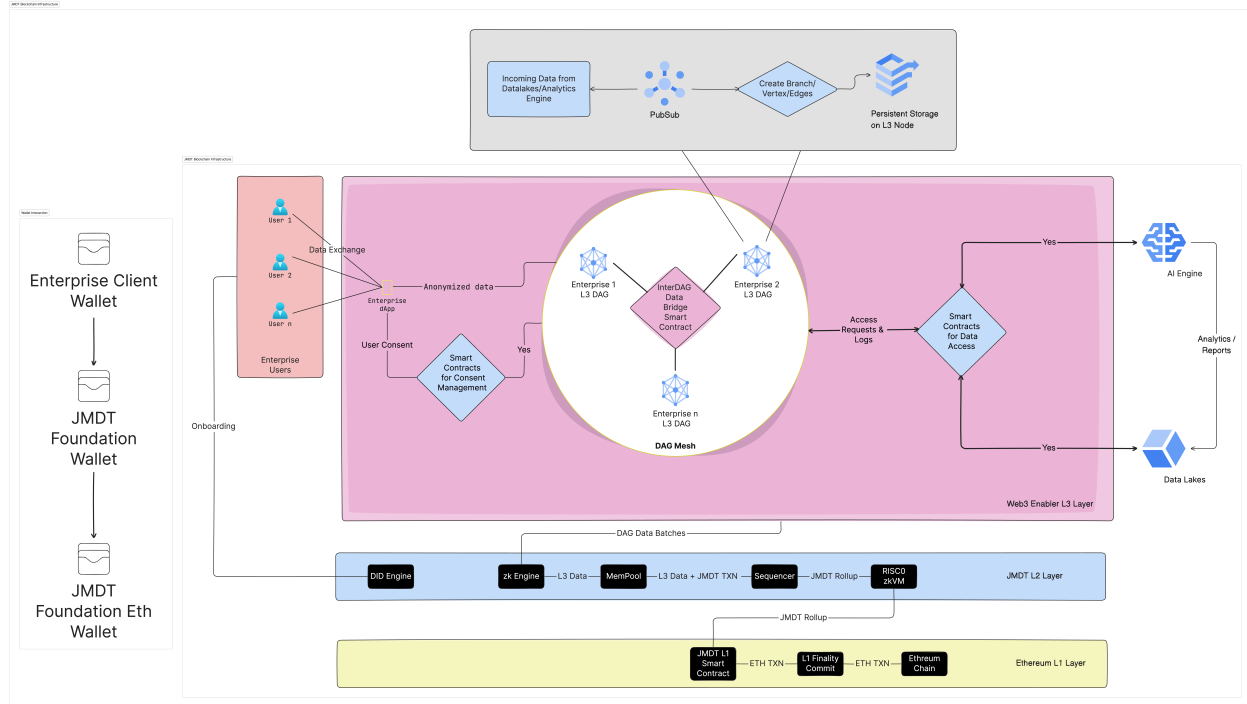
#### 8. Privacy-Preserving Queries

JMDT introduces a **DID-based querying mechanism** that enables **privacy-preserving access to data**. This ensures that:

1. Users and enterprises can query blockchain data **without exposing sensitive details**.
2. **Zero-Knowledge cryptography** prevents unauthorized access while allowing verifiable computations.
3. **Federated and differential privacy techniques** ensure compliance with data protection laws while enabling valuable insights.
4. With these advanced privacy-preserving mechanisms, JMDT fosters a **secure, scalable, and interoperable blockchain ecosystem** for **financial services, enterprise data sharing, and next-generation decentralized applications**.

## 4. Jupiter Meta Data Token Architecture

Jupiter Meta Data Token (JMDT) is a multi-layered blockchain infrastructure designed to support **scalable, privacy-preserving, and enterprise-grade data processing**, from user consent to Ethereum finality. The architecture is divided into three core layers—L3 (enterprise DAGs), L2 (zkRollup + consensus), and L1 (Ethereum anchoring).



## 4.1 Layer 3 – Enterprise DAG & Consent Infrastructure

- **Enterprise DAG Nodes:** Each organization or dApp (e.g., SuperJ, Hercules, gaming, finance, healthcare) operates a private DAG node for high-throughput, localized operations. DAG nodes are synchronized internally using **RAFT consensus**, while state transitions are periodically committed to L2.
- **Smart Contracts for Consent & Access:**
  - Govern user onboarding, consent capture, and access rights for data exchange.
  - Only anonymized data is published to DAGs post-consent.
- **InterDAG Bridge:**
  - Facilitates cross-enterprise collaboration using shared smart contracts.
  - Enables access requests, logging, and secure off-chain queries.
- **Pub/Sub DAG Mesh:**
  - Supports streaming analytics and real-time data ingestion via Pub/Sub architecture.
  - Every node builds **vertices and branches**, logged with **persistent storage**, and synchronized using **RAFT**.

## 4.2 Layer 2 – ZK-Enabled State and Rollup Layer

- **zk Engine (STARK):** Verifies DAG transactions and batch validity using zero-knowledge proofs generated by the RISC Zero zkVM.
- **DID Engine:** Provides W3C-compliant Decentralized Identity, allowing private yet verifiable authentication.

- **AVC Consensus Mechanism:**
    - Combines **VRF-based buddy selection, asynchronous quorum validation, zk-proof enhanced verification, gossip-based propagation, and CRDT-driven conflict resolution.**
    - Ensures **fast and scalable finality, tamper-proof auditability, and high availability** across a decentralized peer network without permanent validator roles.
  - **Sequencer and Mempool:**
    - Aggregates DAG state transitions into zk-proofs using a modular zkVM.
  - **Transaction Type Support:**
    - JMDT's RPC, mempool, orchestrator, and ZK proof pipeline carry EIP-1559 Type 2 transactions end-to-end in production, alongside continued support for legacy Type 0 transactions.
    - Type 2 transactions carry `type (uint8 = 2)`, `max_fee_per_gas`, `max_priority_fee_per_gas`, and `gas_used` as first-class fields, propagated unmodified from mempool ingestion through to L1 commitment.
    - This enables fee-market-aware transaction ordering and predictable gas pricing consistent with Ethereum's post-London fee model.
  - **RISC Zero zkVM:**
    - Executes rollup verification logic as a deterministic, auditable guest program.
    - Outputs STARK proofs submitted to Ethereum for L1 anchoring.
  - **Mempool Routing Engine:**
    - The **Mempool Routing Engine** distributes transactions across a pool of independent mempools via gRPC, allowing horizontal scaling of transaction admission and validation independent of sequencer throughput.
    - This decouples ingestion load from block production and allows mempool instances to be scaled or replaced without sequencer downtime.
- 

### 4.3 Layer 1 – Ethereum Settlement Layer

- **JMDT Smart Contracts:**
    - Receive L2 commitments, verify zkVM proofs, and finalize state on Ethereum.
  - **L1 Finality Commit:**
    - Ensures integrity and auditability of enterprise and validator activity.
    - Supports transparent public settlement while preserving L3 data privacy.
- 

### 4.4 Key Properties

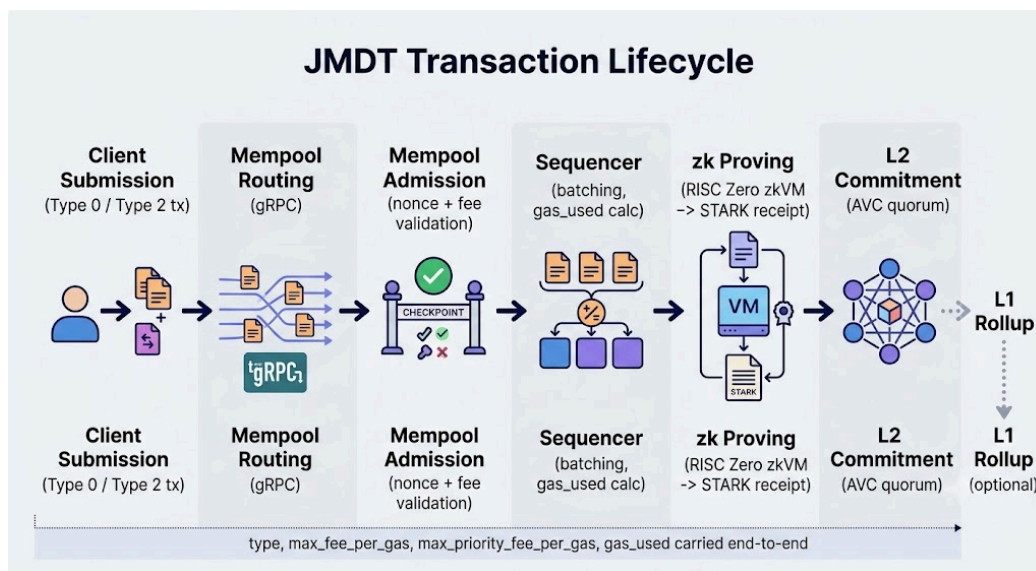
- **Privacy-Preserving:** We use Decentralised Identity verification, Zero-knowledge proof data validation and data sharing - ensuring personal data is verified without even being shared.
  - **Enterprise-First:** The L3 DAG Mesh Network allow private computation, internal tokenization, and industry-specific logic.
  - **Verifiable Rollups:** RISC Zero zkRollups provide immutable, trust-minimized proofs of all activity.
-

## 4.5 Transaction Lifecycle

A transaction on JMDT progresses through the following stages, each of which preserves the transaction's original type and fee fields:

1. **Submission** — A client submits either a legacy (Type 0) or EIP-1559 (Type 2) transaction to a JMDT RPC endpoint.
2. **Routing** — The Mempool Routing Engine assigns the transaction to a mempool instance via gRPC based on load-balancing policy.
3. **Admission** — The assigned mempool validates signature, nonce ordering, and (for Type 2) that `max_fee_per_gas ≥ max_priority_fee_per_gas`.
4. **Sequencing** — The Sequencer selects transactions from across mempool instances and orders them into a batch, computing `gas_used` per transaction.
5. **Proving** — The batch is passed to the zk Engine, where a RISC Zero zkVM guest program validates the batch and produces a STARK receipt (see Section 7).
6. **Commitment** — The resulting commitment is submitted to L2 consensus (AVC) for buddy-quorum validation (see Section 8), then periodically rolled up to Ethereum L1 in batches.

Type (0 or 2), `max_fee_per_gas`, `max_priority_fee_per_gas`, and `gas_used` are preserved as struct fields at every stage — no transaction is normalized or stripped of its original fee-market metadata prior to proving.



## 5. Ethereum as the Foundation Layer

Ethereum serves as the foundational Layer 1 blockchain for JMDT due to its robust smart contract capabilities, security, and decentralized ecosystem. The Ethereum network provides the necessary infrastructure for executing smart contracts, ensuring trustless interactions, and facilitating scalable Layer 2 solutions through rollups.

**Key Benefits of Building on Ethereum:**



- **Security:** Ethereum's battle-tested consensus mechanisms and extensive developer community provide a highly secure environment for decentralized applications.
- **Scalability with Layer 2:** Ethereum's ecosystem supports zk-rollups and other Layer 2 solutions, enabling high transaction throughput with reduced gas costs.
- **Interoperability:** Ethereum's widespread adoption allows JMDT to interact seamlessly with other Web3 protocols, DeFi platforms, and enterprise blockchain applications.
- **Smart Contract Functionality:** Ethereum's Solidity-based smart contracts enable automation and secure execution of decentralized agreements.
- **Sustainability:** With Ethereum's transition to Proof-of-Stake (PoS), energy consumption has been significantly reduced, aligning with JMDT's sustainability goals.

By leveraging Ethereum as its base layer, JMDT ensures that its Layer 2 solution inherits Ethereum's decentralization, security, and interoperability while introducing efficiency and privacy enhancements through Zero-Knowledge Proofs and DID-based identity management.

## 5.1 L1 Commit Protocol

JMDT's rollup commitment to Ethereum is governed by an on-chain contract, `ZKRollup.sol`, and an off-chain orchestrator responsible for gas-aware submission timing.

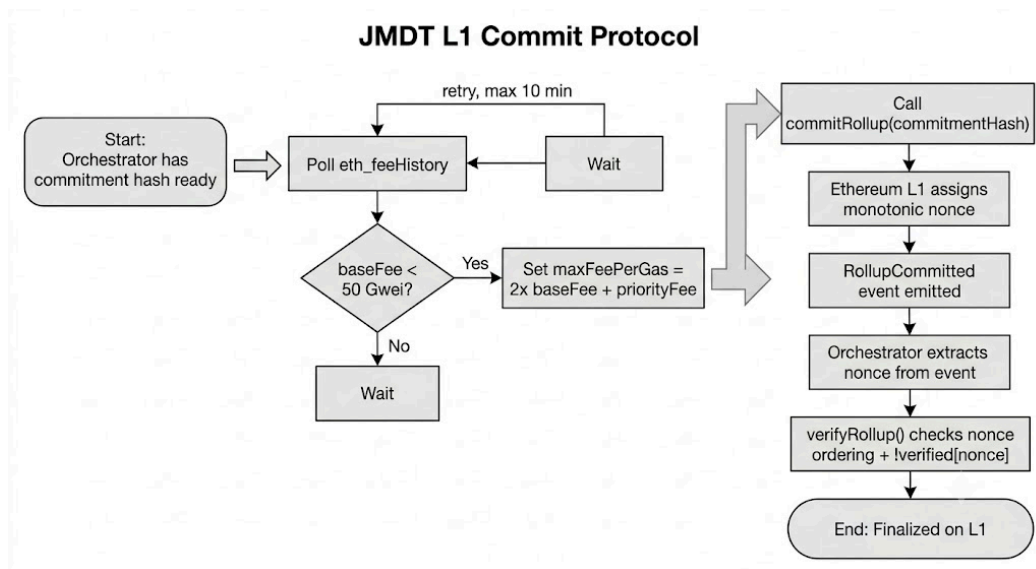
### 5.1.1 On-Chain Contract Behavior

- `commitRollup(bytes32 commitmentHash)` accepts a batch commitment hash and assigns it a **monotonically increasing nonce**.
- `verifyRollup()` enforces strict **nonce ordering** — rollups must be verified in the sequence they were committed — and includes **double-commit protection** via `require(!verified[nonce])`, preventing the same nonce from being verified twice.
- Upon successful commitment, the contract emits `RollupCommitted(uint256 indexed nonce, ...)`. The orchestrator extracts the assigned nonce from this event after the transaction is mined, rather than assuming a client-side nonce value — eliminating race conditions between concurrent orchestrator instances.

### 5.1.2 Gas-Aware Submission (EIP-1559)

The orchestrator does not submit commitments unconditionally. It:

1. Polls `eth_feeHistory` to track current Ethereum base fee.
2. Waits until `baseFee < 50 Gwei` before submitting, to avoid committing during fee spikes.
3. Sets `maxFeePerGas = 2 × baseFee + priorityFee` per submission, consistent with standard EIP-1559 fee-bidding practice.
4. Enforces a **10-minute deadline** — if the base-fee condition is not met within 10 minutes, the orchestrator proceeds with submission regardless, to avoid indefinite delay of finality.



## 6. JMDT Decentralized Identity (DID)

JMDT implements **W3C-standardized Decentralized Identifiers (DIDs)** to provide users with secure, self-sovereign identities. Users with a **JMDT DID** can seamlessly authenticate into decentralized applications (dApps) built on the JMDT Layer without having to share any personally identifiable information (PII) whatsoever.

This DID-based ZKP authentication ensures **privacy, security, and user control** over identity verification.

### 6.1 How Users Create a JMDT DID

- **Onboarding Through Services:** Users can create a **JMDT DID** by registering through a **service provider, dApp, or enterprise system** that integrates JMDT's identity framework.
- **Self-Generated DID:** Advanced users and enterprises can **self-generate** a DID using **cryptographic key pairs as mentioned in JMDT Documentation** and register them on the JMDT network.
- **Enterprise-Managed DIDs:** Organizations can issue **enterprise-controlled DIDs** for employees, partners, or customers, enabling **private, ZK verifiable access** to enterprise services.

### 6.2 Benefits of JMDT DID-Based Authentication

- **PII-Protected Access:** Services utilizing **JMDT DID** can provide **secure, decentralized Identity authentication** without accessing personal data using our on-device Zero Knowledge computation.
- **Interoperability with Web3 & Enterprises:** Users can seamlessly access partnered **blockchain decentralized applications, and enterprise systems** while maintaining full privacy and security.
- **Self-Sovereign Identity:** Users have **full control over their credentials and the data**, reducing reliance on centralized identity providers.

- **Enhanced Security: Zero-Knowledge cryptographic techniques** prevent unauthorized access to user data while allowing verifiable interactions.

---

## 7. Zero-Knowledge Proofs: STARKs and the RISC Zero zkVM

Zero-Knowledge Proofs (ZKPs) are cryptographic techniques that allow one party to prove knowledge of certain information without revealing the information itself. ZKPs ensure privacy and security while maintaining verifiability, making them a cornerstone of blockchain scalability and data integrity.

JMDT's production proving pipeline is built exclusively on RISC Zero's zkVM, which generates STARK proofs — offering transparency (no trusted setup) and post-quantum-resistant, hash-based security guarantees.

---

### 7.1 ZK Circuit Implementation and RISC Zero Integration

Jupiter Meta Data Token employs a two-tier circuit architecture (transaction validation and aggregation), leveraging custom zk-circuits written in **Rust** and executed within the **RISC Zero VM** for high-assurance proofs. This architecture ensures both transaction-level privacy and verifiable state transitions from Layer 3 and Layer 2 to Ethereum Layer 1.

#### Circuit Design in Rust

All ZK circuits run on **RISC Zero zkVM 3.0**, using **local succinct STARK proving** (`ProverOpts::succinct()`), which produces constant-size proofs suitable for on-chain verification regardless of the underlying computation's trace length.

#### **Circuit Types:**

##### 1. Transaction Validation Circuits (L2):

- Written in Rust as deterministic logic.
- Validate balances, DID authentication, signature correctness, and state transitions within L2.
- Output a commitment hash that is consumed by the zk-rollup aggregator.

##### 2. Aggregation & State Transition Circuits (L2 → L1):

- Aggregate multiple rollup blocks or DAG state transitions.
- Execute recursive verification logic or merkle root reconciliation.
- Generate a succinct STARK proof using RISC Zero's zkVM.

#### RISC Zero zkVM for Ethereum Finality

JMDT utilizes the **RISC Zero zkVM** to provide **verifiable computation** for all Ethereum L1 commitments. It acts as a zk-powered virtual CPU that proves execution of arbitrary Rust code.

#### • **Workflow:**

- A `TxnRequest` (serialized as JSON) is passed to the guest ELF binary.
- The zkVM executes the guest program and produces a STARK receipt.

- Field hashes are committed using the scheme described in Section 7.2 (Commitment Security) below.
- The receipt is serialized via `bincode` and returned to the orchestrator for submission to L1.

- **Advantages:**

- No need to define manual R1CS or arithmetic circuits.
- Enables fast iteration, modular upgrade paths, and readable cryptographic logic.
- Guarantees integrity and reproducibility of enterprise workflows and DAG transitions.

## Security and Upgradeability

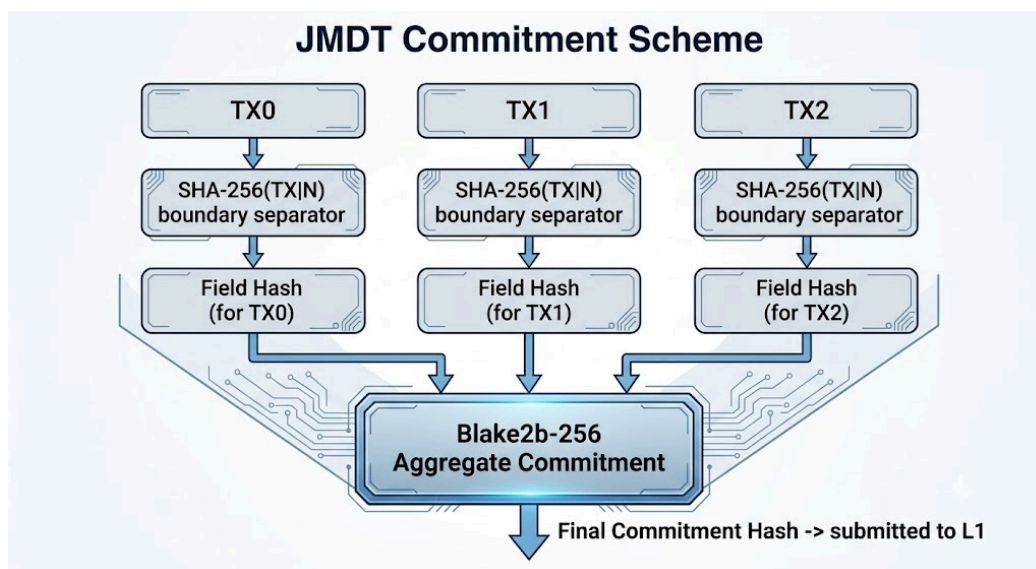
By writing ZK circuits in Rust and compiling them for the zkVM:

- We achieve full type safety, memory safety, and logic reuse.
- Circuits are easy to audit and version-control.
- ZK proofs are consistent across validator nodes and enterprise DAGs, with deterministic reproducibility.

## 7.2 Commitment Security

JMDT utilise a cryptographically secure, replay- and reordering-resistant scheme:

- **Per-transaction boundary separation:** Before hashing a transaction's fields, a SHA-256 boundary separator — `SHA-256("TX|N")`, where `N` is the transaction's index within the batch — is prepended to that transaction's field hash. This binds each transaction's commitment to its position in the batch, preventing an adversary from reordering transactions within a proven batch without invalidating the commitment.
- **Final aggregate commitment:** All per-transaction field hashes (each already bound to its boundary separator) are aggregated into a single **Blake2b-256** hash, forming the batch's final commitment.
- **Security rationale:** `fxhash` is a non-cryptographic hash designed for speed in hash tables, not collision resistance — it was never suitable for commitments an L1 contract relies on for finality. SHA-256 boundary separation plus a Blake2b-256 aggregate gives the commitment scheme both **collision resistance** and **order-binding**, closing a class of reordering attacks that would otherwise be possible against a naive concatenated hash.



---

## 8. Consensus Mechanism: Asynchronous Validation Consensus (AVC) with zkProofs and Peer Quorum

To achieve fast, tamper-resistant, and scalable block finality in the Jupiter Meta Data Token (JMDT) Layer-2 blockchain, we implement **Asynchronous Validation Consensus (AVC)** — a quorum-based, non-synchronous consensus model designed for decentralized environments and strengthened with zk-proof guarantees.

Unlike producer-driven consensus, AVC relies on **collective validation by randomized buddy sets**, ensuring fairness, resilience, and transparency. Nodes independently verify proposed blocks, align on Merkle digests, and reach quorum without relying on fixed committees or synchronous rounds.

---

### 8.1 Key Components

#### Asynchronous Validation

- When the Sequencer proposes a block, **all JMDN nodes immediately enter validation mode**.
- This event-driven, non-blocking design allows **parallel validation** across the network without reliance on global timing or designated leaders.

#### Buddy Set Quorum

- A **VRF-based Buddy Node Selection Algorithm**, weighted by Seed Node feedback, deterministically selects a buddy set of  $k$  nodes each round.
- Buddies perform:
  - Signature and DID checks,
  - Balance sufficiency and ownership validation,
  - zk-proof verification for batch integrity and privacy.
- A block is accepted when  $\geq q\_buddy = \lceil 2k/3 \rceil$  buddies sign and broadcast a **BuddyVote(digest)**. This threshold guarantees overlap between quorums and resilience against Byzantine behavior.

#### zk-Proof Integrity

- Each block must include a **zk-proof** (generated off-chain by a zkVM or prover service).
- zk-proofs validate batched execution correctness and are verified independently by all buddies.
- This ensures **privacy-preserving validation** without exposing internal state transitions.

#### Conflict Resolution

- If buddy digests diverge, **CRDT-based reconciliation** merges results to achieve eventual consistency.
- A **temporary scoped leader** may coordinate the merge, but no permanent authority exists.
- **Bloom filters** prevent replayed or duplicate messages, lowering bandwidth overhead and speeding up block relay.

## Immutable Ledger

- Finalized blocks are committed to the **JMDT Distributed Ledger Layer**, ensuring:
  - **Append-only, tamper-proof history**,
  - **Auditability** for enterprise use cases,
  - Separate storage of flagged or invalid transactions for accountability.

## 8.2 Security and Operational Benefits

| Feature                                                                                                     | Benefit                                                       |
|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
|  zk-proof validation       | Privacy-preserving and verifiable transaction proofs          |
|  VRF + quorum              | Randomized, fair buddy sets; strong Byzantine fault tolerance |
|  Gossip + Bloom            | Efficient, low-latency peer-to-peer communication             |
|  CRDT-based reconciliation | Convergent state even under partitions                        |
|  Incentive alignment       | Sequencer rewarded only after quorum validation succeeds      |
|  Adaptive weighting        | Seed Nodes penalize malicious nodes and reward honest ones    |

## 8.3 Why This Matters

The **AVC protocol** ensures that block finality in the JMDT ecosystem is:

- **Verifiable** through zk-proofs
- **Decentralized** through randomized buddy-set validation
- **Responsive** with asynchronous quorum instead of synchronized rounds
- **Audit-ready with distributed, append-only ledger records secured through cryptographic verification and consensus**
- **Adaptive** via dynamic weight updates that favor reliable nodes

This design also forms the foundation for **Layer-3 DAG extensions** and enterprise-specific consensus frameworks, making AVC a **future-proof consensus model** for data-driven blockchain applications.

## 9. Peer Sync Protocol (JMDN-FastSync)

Node synchronisation on the JMDT Decentralised Node (JMDN) network follows a staged protocol, **JMDN-FastSync**, comprising eight distinct phases. Each phase is independently resumable and designed to bound both memory usage and time-to-sync for nodes joining or catching up to the network.

| Phase | Name                             | Function                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0     | <b>Availability</b>              | Handshake between peer nodes, UUID-based auth token exchange, and advertisement of each node's available block range.                                                                                                                                                                                                                                                                                                                    |
| 1     | <b>PriorSync</b>                 | Uses MMR (Merkle Mountain Range) accumulator bisection to identify the specific block ranges where local and remote chain state diverge. Emits heartbeats every 5 seconds during long-running bisection computation to signal liveness.                                                                                                                                                                                                  |
| 1a    | <b>Merkle</b>                    | On-demand sub-range Merkle tree requests issued during PriorSync bisection, used to narrow divergence to smaller ranges without transferring full block data.                                                                                                                                                                                                                                                                            |
| 2     | <b>HeaderSync</b>                | Concurrent batch fetching of block headers ( $\leq 1,500$ headers per request, 3 parallel workers, with failover on worker failure). Uses streaming heartbeats and a <code>SyncConfirmation</code> loop capped at 4 rounds to confirm header-set agreement between peers.                                                                                                                                                                |
| 3     | <b>DataSync</b>                  | Concurrent batch fetching ( $\leq 30$ blocks per request) of full transaction data, ZK proofs, and L1 finality metadata for the confirmed header set.                                                                                                                                                                                                                                                                                    |
| 4     | <b>PoTS (Point-of-Time-Sync)</b> | Reconciles blocks produced <i>during</i> phases 1–3 (buffered in a write-ahead log while the main sync was in progress) once the main sync completes, ensuring no blocks produced mid-sync are lost.                                                                                                                                                                                                                                     |
| 5     | <b>AccountSync</b>               | Bidirectional streaming reconciliation of account state. The client uploads ART (Adaptive Radix Tree)-encoded account nonce chunks; the server diffs these against its full account set using a <b>SwappableART</b> structure (memory-bounded, spilling to disk once account count exceeds 1M keys) and streams back missing accounts in parallel pages, terminated by an <code>EndOfStream</code> marker for completeness verification. |
| —     | <b>PubSub</b>                    | Runs continuously alongside the above (not a discrete phase): GossipSub-based block announcements keep already-synced, live nodes updated in real time.                                                                                                                                                                                                                                                                                  |

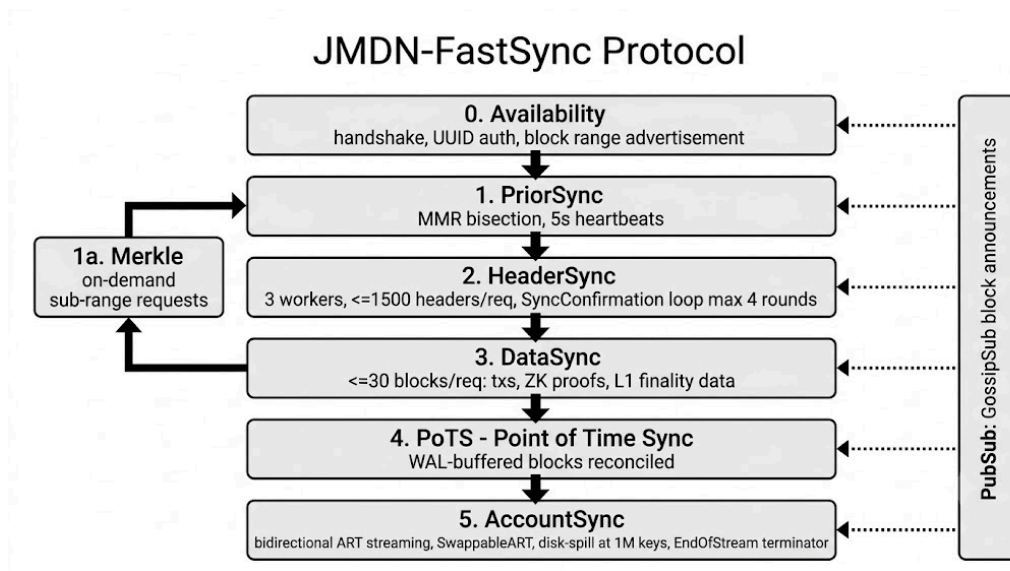
## 9.1 Reconciliation & Node Catchup

### 9.1.1 Delta-Based Reconciliation

Post-sync account reconciliation avoids full-history replay. Using `GetTransactionsForAccountInRange(from, to)`, only accounts that were actually touched within the synced block range are replayed against local state — accounts with no activity in that range are left untouched. This bounds reconciliation cost to the volume of *changed* state rather than total chain history.

### 9.1.2 Node Catchup

Once a node completes initial JMDN-FastSync, it enters a **catchup** phase to absorb blocks produced after the original sync window closed. Catchup is governed by two configurable parameters, `CatchUpFromBlock` and `CatchUpPeer`, and reuses the existing **HeaderSync** → **DataSync** → **ReconcileWithDeltas** pipeline rather than introducing a separate code path — ensuring catchup and initial sync share the same correctness guarantees.



### 9.1.3 Node Onboarding

- A new JMDN node joining the network follows the full JMDN-FastSync sequence from Phase 0 (Availability) through Phase 5 (AccountSync), then transitions into steady-state operation via the Node Catchup pipeline to close any gap accrued during onboarding, before subscribing to the PubSub GossipSub channel for live block propagation.
- This shared pipeline design — onboarding, catchup, and steady-state sync all built from the same HeaderSync/DataSync/reconciliation primitives — reduces the surface area for divergent bugs between "a node joining for the first time" and "a node that briefly went offline," which are otherwise common sources of chain-state inconsistency in distributed systems.

## 10. Security Model

The security of the **JupiterMeta Data Token (JMDT)** protocol is anchored on a layered approach involving **cryptographic guarantees** and **ZK-based verifiability** across L3, L2, and L1 layers.

### 10.1 Trust and Threat Assumptions

| Layer                      | Trust Model                                                                                       | Threats Mitigated                               |
|----------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <b>L1 (Ethereum)</b>       | Assumes Ethereum finality and censorship-resistance                                               | Network reorgs, malicious smart contracts       |
| <b>L2 (JMDT Chain)</b>     | Assumes $\geq 2/3$ honest, randomized and democratized validators in AVC                          | Sybil attacks, block manipulation, equivocation |
| <b>L3 (Enterprise DAG)</b> | Assumes internal integrity within enterprise                                                      | Insider tampering, unauthorized data access     |
| <b>ZK Proof System</b>     | All L2 and L3 state transitions are validated via zk-STARK proofs generated by the RISC Zero zkVM | Proof forgery, data leakage                     |



| Layer          | Trust Model                                | Threats Mitigated                     |
|----------------|--------------------------------------------|---------------------------------------|
| RISC Zero zkVM | Assumes deterministic Rust guest execution | Non-determinism, circuit manipulation |

## 10.2 Security Guarantees

### 1. Integrity:

- All L2 and L3 state transitions are validated via **zk-STARK proofs** generated by the **RISC Zero zkVM**.
- Transactions are only finalized if proofs are verified and consensus is reached.

### 2. Confidentiality:

- Personally identifiable information (PII) is never committed to-chain.
- DID-based authentication ensures access control without revealing identity.
- L3 DAGs keep enterprise data off-chain; only zk-committed proofs reach L2.

### 3. Availability:

- L2 consensus leverages **Gossip Protocol** for fast propagation.
- DAGs are internally redundant and support CRDT-based recovery.

### 4. Accountability:

- All block proposals and zk-proofs are attributable to known DID identities or validator keys.
- Invalid or conflicting proofs are rejected automatically by L2 validators.

## 10.3 Potential Attack Surfaces

| Vector                         | Mitigation                                                                 |
|--------------------------------|----------------------------------------------------------------------------|
| Sybil attacks on validator set | Unpredictable Randomized Validator Sets + DID verification                 |
| zk-circuit manipulation        | Rust-based zkVM ensures reproducible, deterministic execution              |
| Data leakage in queries        | All data access is gated via DID + ZK queries with no plaintext exposure   |
| L3 DAG integrity compromise    | DAG nodes must commit proofs to L2; failures are detected and slashed      |
| Delayed L1 commitments         | Dynamic L1 batching policy ensures gas-free aware but time-bounded commits |
| Smart contract bugs            | All Ethereum contracts are audited and open-sourced                        |

## 10.4 Formal Verification and Auditing

- All zkVM guest programs undergo **formal verification** for determinism and safety.
- The **JMDT Decentralised Node (JMDN)** implementation has completed an independent security audit and certification process. Report available at

[https://github.com/JupiterMetaLabs/jmdn/blob/1197f5aec7ecd0f7126481771792dd44b714afb2/audits/2026-03-terasoft-certin-vapt/TERA\\_CERT-IN\\_03\\_2026\\_CR\\_16\\_Certificate.pdf](https://github.com/JupiterMetaLabs/jmdn/blob/1197f5aec7ecd0f7126481771792dd44b714afb2/audits/2026-03-terasoft-certin-vapt/TERA_CERT-IN_03_2026_CR_16_Certificate.pdf)

- The DID module conforms to **W3C standards** and supports verifiable credential flows.

This security model ensures **robust protection across privacy, integrity, and availability dimensions** while maintaining performance, scalability, and compliance.

## 11. Tokenomics & Incentives

JMDT has a fixed maximum supply of **1,000,000,000 (1 Billion) tokens**. The token economy is designed to balance ecosystem growth, decentralisation, long-term sustainability, research and innovation, community participation, strategic capital formation, and market liquidity.

The token allocation framework has been further refined to provide greater transparency, align incentives across all stakeholders, and support the long-term development of the JMDT ecosystem.

### 11.1 Token Allocation

| Category                           | Allocation  | Tokens               |
|------------------------------------|-------------|----------------------|
| Ecosystem Incentives               | 27%         | 270,000,000          |
| Foundation / DAO Treasury          | 15%         | 150,000,000          |
| Advisors                           | 2%          | 20,000,000           |
| Community Distribution             | 11%         | 110,000,000          |
| R&D & Public Goods                 | 10%         | 100,000,000          |
| Founding Team & Early Contributors | 10%         | 100,000,000          |
| Founders                           | 5%          | 50,000,000           |
| Strategic Seed Investors           | 15%         | 150,000,000          |
| Liquidity & Market Operations      | 5%          | 50,000,000           |
| <b>Total Supply</b>                | <b>100%</b> | <b>1,000,000,000</b> |

### 11.2 Vesting & Unlock Framework

To ensure long-term alignment and protect the sustainability of the ecosystem, token allocations are subject to structured vesting and release schedules.

#### Team, Founders, Advisors and Strategic Investors

- Subject to multi-year vesting schedules.
- Cliff periods and linear unlock mechanisms may be applied based on governance-approved frameworks.
- Designed to align participants with the long-term success of the protocol.

#### Ecosystem and Community Allocations

- Released progressively based on ecosystem growth, validator participation, network adoption, governance initiatives, and strategic ecosystem programs.
- Distribution is designed to reward meaningful participation and sustainable network expansion.

### Treasury and R&D Allocations

- Deployed through governance-controlled processes and Foundation-approved initiatives.
- Focused on protocol development, ecosystem sustainability, public goods funding, and long-term innovation.

## 11.3 Token Utility

The JMDT token serves as the native utility asset of the ecosystem and may be utilized for:

- Network participation and staking.
- Validator and ecosystem incentives.
- Governance and voting mechanisms.
- Layer 2 and Layer 3 ecosystem operations.
- Enterprise and decentralized application integrations.
- Access to protocol services and infrastructure.
- Future ecosystem utility approved through governance processes.

The tokenomics framework is designed to support a sustainable, decentralized, and utility-driven ecosystem while aligning incentives among users, developers, enterprises, validators, contributors, and long-term stakeholders.

## 12. How Enterprises build on JMDT Blockchain Stack

### 1. Choose Your Enterprise Layer

- **L3 DAG Network:** Enterprises deploy private, high-throughput Directed Acyclic Graph (DAG) nodes for internal data operations (e.g., user activity, insights, compliance logs).
- Each L3 node is permissioned, independently scalable, and **anchored to JMDT Layer 2** for rollup-based finality and zk-proof auditability.

### 2. Set Up Enterprise DID & Access

- Register a **Decentralized Identifier (DID)** representing the enterprise and link sub-identities for departments or teams.
- Manage keys using secure wallets.

### 3. Deploy Custom Workflows on DAG

- Build DAG-based business logic in Python or Rust—e.g., for supply chains, compliance, CRM, or AI-inference logs.

- Use the DAG SDK and APIs to track state transitions and hash each operation.

#### 4. Rollup to Layer 2 (JMDT Chain)

- Periodically batch DAG transactions and submit zk-rollups to L2 using zk-STARK proofs generated via RISC Zero's zkVM.
- JMDT tokens are used to commit rollups, and **Ethereum L1 finality** is optionally supported.

#### 5. Use JMDT for Operations

- **Jupiter Meta Data Token (JMDT)** fuels DAG rollups, L2 commitments, and node staking.
- Enterprises receive JMDT via private allocation, grants.

#### 6. Run or Choose Infrastructure

- Deploy L3 DAG nodes on public cloud, private cloud, hybrid cloud, or on-premises environments while maintaining interoperability with the **JMDT Layer 2 network**, RPC infrastructure, and decentralized storage services.
- Utilize sequencers, relayers, and gateway nodes to securely transmit batched transactions and proofs from Layer 3 to Layer 2.
- Benefit from enterprise-grade deployment flexibility, allowing organizations to retain sovereignty over their infrastructure, data, and operational policies.
- Leverage cryptographic verification, immutable ledger guarantees, and distributed state synchronization across participating nodes.

#### 7. Monitor & Reward

- Use **Explorer dashboards** to visualise DAG graphs, participation, rollup health, and proof verification.
- Trigger user or team-level rewards via smart contracts or DID-linked incentive programs.

---

### 12.1 Layer 3s on JMDT Blockchain

#### Hercules

Hercules is a research assistant for data-backed decision making.

#### SuperJ

A decentralised consumer intelligence app that currently allows over 20 millions users to ZK-verify their identity, share data and monetise information.

---

## 13. Comparative Analysis

The following table compares **Jupiter Meta Data Token (JMDT)** with leading L2 blockchain infrastructure projects across key dimensions relevant to both enterprise and developer adoption.

| Feature / Platform       | JupiterMeta Data Token (JMDT)      | Polygon zkEVM                  | Starknet                                  | Aztec Network                 | Hyperledger Fabric      |
|--------------------------|------------------------------------|--------------------------------|-------------------------------------------|-------------------------------|-------------------------|
| Layer Type               | L2 + L3 (Enterprise DAG)           | L2 (zkEVM)                     | L2 (STARK Rollup)                         | L2 (Private zk-SNARKs)        | Private Consortium      |
| Scalability (TPS)        | 2,000+ TPS (L2); 10K+ (L3 DAG)     | 2,000–4,000 TPS                | 3,000+ TPS                                | <100 (privacy bottleneck)     | Depends on config       |
| Privacy Model            | zk-SNARK + DID + zkVM              | zk-SNARKs                      | zk-STARKs (public)                        | zk-SNARKs (Private Tx)        | No native privacy       |
| zk Circuit Strategy      | Rust + RISC Zero zkVM              | Circom + Groth16               | Cairo VM                                  | Circom + Aztec Noir           | None                    |
| Finality Time (L2)       | ~3–10 sec                          | ~5–10 min                      | ~10–15 min                                | ~30–60 min                    | Instant (centralized)   |
| L1 Settlement (Ethereum) | Dynamic zk-rollup to L1            | Periodic zk-rollup             | STARK rollup                              | Delayed zk-rollup             | Not applicable          |
| Data Availability        | zk-anchored + immutable audit logs | Ethereum DA Layer              | On-chain calldata                         | On-chain                      | Off-chain (private)     |
| Enterprise Readiness     | ✅ DAG layer, DID, privacy          | ⚠️ Limited DID/enterprise SDKs | ⚠️ Limited identity & compliance features | ❌ Not enterprise-ready        | ✅ Proven enterprise use |
| Token Model              | JMDT (Governance + Utility)        | MATIC (Gas & Staking)          | STRK                                      | AZTEC                         | None (no token)         |
| Smart Contract Support   | Rust (zkVM) + Solidity             | Solidity (zkEVM)               | Cairo                                     | Aztec Noir + limited Solidity | Chaincode (Go/Java)     |
| Governance               | Foundation                         | Off-chain + Foundation         | TBD (future DAO)                          | Aztec Foundation              | Centralized consortium  |

## Key Takeaways:

- **JMDT** uniquely combines **scalability**, **privacy**, and **enterprise DAGs**, with **AVC consensus** and **zkVM-based Ethereum finality**.
- It bridges the gap between **public zk-rollups** (like Starknet) and **enterprise-grade blockchains** (like Hyperledger).
- The inclusion of **Decentralised Identity (DID)** and **privacy-preserving queries** gives JMDT a clear advantage for GDPR-compliant and audit-friendly use cases.

## 14. Development Roadmap

JMDT's development follows a seven-phase roadmap spanning research and protocol design through long-term AI-attestation infrastructure. As of this document's publication, **Phases 1 through 3 are complete**: the

token is live and tracked on CoinMarketCap, CoinGecko, and BitMart; mainnet is operational; and ZK proofs, Decentralized Identity, and AVC consensus are running in production.

## Phase 1 — Research & Foundation

Q1–Q2 2024 ·  Complete

Laying the cryptographic and architectural groundwork before a single line of node code shipped.

- **ZK Proof Research** — Deep-dive into ZK-SNARK/STARK primitives; selection of the proof system suited for L2 throughput and on-chain verification cost.
  - **DID Architecture Design** — W3C-compliant Decentralized Identity framework; key schema, resolution protocol, and revocation model.
  - **AVC Consensus Design** — Asynchronous Validation Consensus with BFT safety, BLS aggregate signatures, and VRF-based leader election.
  - **Immutable Ledger Selection** — Evaluation and adoption of ImmuDB as the append-only, tamper-proof backing store for the JMDN node.
  - **JMDN Node Prototype** — First internal build of the Go-based Layer 2 node with P2P messaging via libp2p and Yggdrasil.
  - **ZKP-Based Data Querying** — Prototype DID-keyed queries where query proofs are generated and verified without exposing raw data.
- 

## Phase 2 — Testnet & Protocol Hardening

Q3 2024 – Q1 2025 ·  Complete

Taking the protocol from internal prototype to a live public testnet under real network conditions.

- **JMDT Testnet Launch** — Public testnet deployed on Ethereum Sepolia; open to external validators and developers.
  - **ZK-Rollup Aggregator** — Batch transaction aggregator compressing L2 state transitions into a single ZK proof posted to Ethereum L1.
  - **JMDN FastSync Protocol** — Eight-protocol P2P sync engine (Merkle bisection, header sync, data sync, PoTS, account sync) for new nodes joining the network.
  - **DID Registry On-Chain** — Live DID registration, resolution, and reference linking on testnet; compliant with the W3C DID Document standard.
  - **Mempool & Transaction Pipeline** — Production mempool with nonce management, duplicate detection, and fee prioritization.
  - **Beta Enterprise Testing** — Closed beta with select enterprise partners validating DID-based data queries and ZK attestation flows.
  - **Security Audits (Round 1)** — Independent audit of consensus logic, ZK circuit implementations, and P2P message handling.
- 

## Phase 3 — Mainnet Launch & Market Presence

Q2–Q4 2025 ·  Complete

Production launch, token distribution, and first exchange presence.

- **JMDT Mainnet Launch** — Full production network live; JMDN nodes operating with AVC consensus and ZK proof submission to Ethereum L1.
  - **20M Verified User Capacity** — DID registry architected and tested to handle 20M+ verified identities.
  - **CoinMarketCap Listing**  Completed
  - **CoinGecko Listing**  Completed
  - **BitMart CEX Listing** — Listed 9 December 2025.  Completed
  - **JMDT Documentation Portal** — Public developer docs at docs.jmdt.io covering node setup, RPC API, DID spec, and ZK proof integration.
  - **Block Explorer V1** — Live explorer for blocks, transactions, accounts, and DID records.
- 

## Phase 4 — Ecosystem Foundation

Q1–Q4 2026 ·  In Progress

Turning a live chain into a developer platform with real liquidity and tooling.

- **EIP-1559 Type 2 Transaction Support** — Production support across RPC, mempool, orchestrator, and ZK proof pipeline, alongside continued legacy Type 0 support. 
  - **RISC Zero zkVM 3.0 Upgrade** — Local succinct STARK proving ( `Prover0pts::succinct()` ) for constant-size, on-chain-verifiable proofs.  Completed
  - **Commitment Security Overhaul** — SHA-256 boundary separation plus Blake2b-256 aggregate commitment, replacing the non-cryptographic `fxhash` scheme; closes transaction-reordering attack surface.  Completed
  - **L1 Commit Protocol Hardening** — `ZKRollup.sol` nonce-ordering enforcement and double-commit protection ( `require(!verified[nonce])` ); EIP-1559 gas-aware submission via the orchestrator.  Completed
  - **Full EVM Compatibility** — Deploy any Solidity smart contract on JMDT L2 without modification; pass Ethereum's standard EVM test suite.
  - **JMDT ↔ Ethereum Mainnet Bridge** — Two-way asset bridge with ZK-proof-based fraud proofs; no centralised relayer.
  - **Block Explorer V2** — Real-time explorer for blocks, transactions, accounts, DID records, ZK proofs, and smart contracts.
  - **CEX Expansion** — Target 2–3 additional centralised exchange listings to improve liquidity and price discovery.
  - **Community Bug Bounty Program** — Structured vulnerability disclosure program covering core node software, smart contracts, and bridge infrastructure, with tiered rewards scaled to severity.
- 

## Phase 5 — DID & ZK Product Layer

Q1–Q2 2027 ·  Planned

Turning the "Truth Layer" positioning into products enterprises and users actually ship with.

- **Verifiable Credential Marketplace** — Issue, hold, and verify W3C-compliant credentials on-chain; revocation via ZK proofs without exposing PII.

- **ZK-VM (Zero-Knowledge Virtual Machine)** — Execute private smart contracts where inputs stay hidden; on-device proving at the application layer.  Completed
- **Verifiable Data Oracles** — On-chain attestation of real-world data (KYC outcomes, compliance checks, IoT readings) with ZK proof of source integrity.
- **Enterprise KYC/AML Module** — Prove a user passed compliance checks without exposing underlying identity data to the chain or counterparties.
- **Mobile Wallet** — iOS and Android wallet with native DID management, credential storage, and on-device ZK-proof generation.

## Phase 6 — Multi-Chain & Governance

Q3-Q4 2027 ·  Planned

Expanding reach across the broader ecosystem and decentralizing protocol control.

- **Cross-Chain Bridges** — LayerZero and/or IBC integration; JMDT assets and DID credentials portable to Ethereum, Solana and Tether.
- **Native DEX on JMDT** — AMM-based decentralized exchange; LP incentives funded from protocol fee revenue.
- **Ecosystem Fund** — Grants, hackathons, and university research partnerships to grow the developer base.

## Phase 7 — AI + Truth Layer Applications

2028 ·  Planned

The long-term thesis: JMDT as the verification backbone for AI-generated content and autonomous agents.

- **AI Content Attestation** — On-chain provenance for AI-generated media; ZK proof that content was produced by a verified model or entity.
- **Agent DID Framework** — Autonomous AI agents receive DIDs; every action is signed and permanently auditable on-chain.
- **Truth Score API** — Public API for querying the authenticity score of any data anchored to JMDT; usable by media, finance, and legal sectors.
- **JMDT Data Marketplace** — Privacy-preserving marketplace where verified users monetize personal data with ZK consent proofs; enterprises pay in JMDT.

## Full Timeline

| Phase | Period          | Theme                                                    | Status                                                                                        |
|-------|-----------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 1     | Q1-Q2 2024      | Research, ZK design, DID architecture, node prototype    |  Complete |
| 2     | Q3 2024-Q1 2025 | Testnet, ZK-rollup aggregator, FastSync, enterprise beta |  Complete |
| 3     | Q2-Q4 2025      | Mainnet launch, CMC/CoinGecko/BitMart listings           |  Complete |



| Phase | Period     | Theme                                                             | Status                                                                                         |
|-------|------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| 4     | Q1-Q4 2026 | Protocol Hardening, Fee-Market Upgrade, EVM bridge, CEX expansion |  In Progress |
| 5     | Q1-Q2 2027 | Verifiable credentials, ZK-VM, enterprise KYC/AML                 |  Planned     |
| 6     | Q3-Q4 2027 | Multi-chain, DEX, L3 app-chains, ecosystem fund                   |  Planned     |
| 7     | 2028       | AI attestation, agent DIDs, Truth Score API                       |  Planned     |

## 15. Conclusion

JupiterMeta Data Token (JMDT) represents a paradigm shift in blockchain infrastructure, combining **privacy, scalability, and decentralisation** into a single, seamless ecosystem. By integrating **Zero-Knowledge Proofs (ZKPs), Decentralised Identity (DID), and AVC consensus mechanism**, JMDT ensures **privacy-enhanced, high-performance, and verifiable transactions** across diverse use cases.

JMDT's **Layer 2 (L2) framework** optimises transaction throughput through **zk-rollups**, enabling **cost-effective and scalable interactions** while committing transactions to **Ethereum (L1) dynamically** based on the best available gas fees, block time, and network conditions. The **Layer 3 (L3) DAG architecture** provides a **high-throughput enterprise solution**, allowing businesses to manage **private, off-chain transactions with cryptographic integrity** while anchoring verifiable state proofs on L2.

JMDT also brings **enterprise-grade security and compliance** through **DID-based authentication**, ensuring **secure, trustless identity verification** without exposing personal data.

The **AVC consensus model**, integrating **Bloom Filters**, and **asynchronous quorum validation**, enables rapid and fault-tolerant block propagation. It minimises communication overhead, supports efficient peer validation without global coordination, and achieves robust state convergence through **zk-proof-backed validation** across decentralised buddy nodes.

With **Ethereum interoperability, zk-proof-based privacy preservation, and tokenomics-driven governance**, JMDT fosters an ecosystem that supports:

- **DeFi innovation** with privacy-focused smart contracts and optimised transaction finality.
- **Web3 data marketplaces** that enable **secure, private, and decentralised data exchange**.
- **Enterprise applications** that require **confidentiality, high throughput, and regulatory compliance**.
- **Decentralised identity solutions** that empower users with **sovereign control over their credentials**.

The **future of blockchain** demands a balance between **efficiency, privacy, and interoperability** and JMDT delivers on all fronts. By pioneering **scalable zero-knowledge rollups, enterprise DAG integration, and DID-based access control**, JMDT is poised to become **the backbone of next-generation decentralised infrastructure**.

For collaborations and inquiries, visit <https://jmdt.io>

## 16. References

### Zero-Knowledge Proving System

1. RISC Zero. *zkVM Overview*. RISC Zero Developer Docs. <https://dev.risczero.com/api/zkvm/>
2. RISC Zero. *zkVM Technical Specification*. RISC Zero Developer Docs. <https://dev.risczero.com/api/zkvm/zkvm-specification>
3. RISC Zero. `risc0-zkvm` Rust crate documentation. <https://docs.rs/risc0-zkvm/>
4. RISC Zero. *risc0 — Zero-Knowledge Verifiable General Computing Platform* (GitHub repository). <https://github.com/risc0/risc0>
5. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M. *Scalable, transparent, and post-quantum secure computational integrity* (the original STARK paper). IACR ePrint Archive, 2018/046. <https://eprint.iacr.org/2018/046>

### Ethereum & Transaction Standards

1. Buterin, V., Conner, E., Dietrichs, R., Frame, M., Landsman, D. *EIP-1559: Fee market change for ETH 1.0 chain*. Ethereum Improvement Proposals. <https://eips.ethereum.org/EIPS/eip-1559>
2. Ethereum Foundation. *EIP-1559 FAQ*. <https://notes.ethereum.org/@vbuterin/eip-1559-faq>

### Decentralised Identity

1. W3C Decentralized Identifier Working Group. *Decentralized Identifiers (DIDs) v1.0*. W3C Recommendation. <https://www.w3.org/TR/did-1.0/>
2. W3C Decentralized Identifier Working Group. *Decentralized Identifiers (DIDs) v1.1*. W3C Candidate Recommendation Snapshot, March 2026. <https://www.w3.org/TR/did-1.1/>
3. W3C. *Decentralized Identifier Resolution (DID Resolution) v0.3*. <https://www.w3.org/TR/did-resolution/>

### Cryptographic Hash Functions

1. National Institute of Standards and Technology (NIST). *FIPS 180-4: Secure Hash Standard (SHS)* — defines SHA-256. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>
2. Aumasson, J-P., Neves, S., Wilcox-O'Hearn, Z., Winnerlein, C. *BLAKE2: Simpler, Smaller, Fast as MD5*. Applied Cryptography and Network Security (ACNS), 2013. <https://www.blake2.net/>

### Consensus, Networking & Data Structure Primitives

1. Micali, S., Rabin, M., Vadhan, S. *Verifiable Random Functions*. IEEE Symposium on Foundations of Computer Science (FOCS), 1999. — theoretical basis for VRF-based buddy node selection.
2. Ongaro, D., Ousterhout, J. *In Search of an Understandable Consensus Algorithm (Raft)*. USENIX Annual Technical Conference, 2014. <https://raft.github.io/raft.pdf> — basis for intra-DAG node synchronization (Section 4.1).
3. Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M. *Conflict-Free Replicated Data Types*. Stabilization, Safety, and Security of Distributed Systems (SSS), 2011. — basis for CRDT-based conflict resolution

(Section 8).

4. libp2p. *GossipSub v1.1 Specification*.  
<https://github.com/libp2p/specs/blob/master/pubsub/gossipsub/gossipsub-v1.1.md> — basis for the PubSub block-announcement layer (Section 9).
5. Todd, P. *Merkle Mountain Ranges*. OpenTimestamps / Peter Todd notes.  
<https://github.com/opentimestamps/opentimestamps-server/blob/master/doc/merkle-mountain-range.md>  
— basis for the PriorSync MMR accumulator bisection protocol (Section 9).
6. Bloom, B. H. *Space/Time Trade-offs in Hash Coding with Allowable Errors*. Communications of the ACM, 1970. — basis for Bloom filter-based replay protection.

---

Document Version: 1.5 | July 2026

© 2026 JMDT | Jupiter Meta Labs Foundation | Seychelles | DMCC, UAE |

Github - <https://github.com/JupiterMetaLabs/jmdn>

